

Highly Dependable Systems 2025/2026 - DepChain

Martin Silveira^{1,2[106869]}, Tomás Gomes^{1,3[112307]}, and Tomás Matos^{1,4[117340]}

¹ Institute Superior Técnico, University of Lisbon, Portugal

² `martin.silveira@tecnico.ulisboa.pt`

³ `tomasldgomes2002@tecnico.ulisboa.pt`

⁴ `tomasmbmatos@tecnico.ulisboa.pt`

1 Introduction

The current report aims to describe the details of the design and implementation of the DepChain project that is a simplified permissioned (closed membership) blockchain system with high dependability guarantees, implemented in Java. We follow the simplifying assumptions that this system's membership is static for the entire system lifetime and there is a Public Key Infrastructure in place, meaning that blockchain members should use self-generated public/private keys, which are pre-distributed before the start of the system, and this information is also included in the static system membership.

2 Design

For the design of the system, the group followed the paper related to the HotStuff protocol, that is a leader-based Byzantine fault-tolerant replication protocol for the partially synchronous model [1]. Following the implementation of the consensus the group then developed the contract layer, where blocks comprised of transactions are committed to a blockchain, these transactions are created by transfer requests made by clients, the currency used for the contracts, ISTCoin was made based on the ERC-20 standard [4].

2.1 Network Layer

For the abstraction of the network layers, we implemented the concepts shown in the theoretical classes being them the **Fair Loss Links**, the **Stubborn Loss Links** and finally the **Authenticated Perfect Links**. The natural order would be to also implement the **Perfect Links** in between the **SLL** and **APL** but because the final objective of the network layer was to send and receive authenticated messages the group saw no need to implement PL as this transports non-authenticated messages. So, following the lecturers Miguel Matos and Paolo Romano [2] we have that:

- **Fair Loss Links (FFL)** are a low-level abstraction of the UDP protocol and since this protocol matches the specification of FFL, we simply initialize the UDP socket at this layer.
- **Stubborn Links (SL)** layer sits right on top of the FFL. This is a "persistent" layer when every time a message pops up from the FLL, the SL layer kicks off a timer and it will keep handing that same message off to the layer above it every second, and it won't quit until it's told to stop.
- **Authenticated Perfect Link (APL)**. This layer handles is built on top of SL and has four types of authenticated messages: handshake request messages, handshake acknowledgment messages, acknowledgment messages, client messages and consensus messages. For the handshake between two processes, we create a Diffie-Hellman key pair in order to create their session key. The first process begins the handshake and the second, upon receiving it, will verify the signature of the handshake and generate the session key which will be used for the MAC sent in the handshake ACK. If the MAC verification on the session key succeeds, the handshake is successful and the connection is established. Then, client and consensus messages can be sent between the two processes using this link.

2.2 Consensus

For the consensus layer, we implemented the Basic HotStuff protocol, a leader-based Byzantine Fault Tolerant (BFT) replication algorithm designed for the partially synchronous model.

To reduce the message size and verification overhead, our implementation uses Threshold Signatures (*threshsig*). Instead of appending $n - f$ individual signatures to prove a consensus milestone, the leader mathematically aggregates $n - f$ partial signature shares (**SigShare**) into a single, fixed-size Quorum Certificate (**QuorumCertificate** or **QC**).

The protocol progresses through views, with a rotating leader dynamically calculated for each view (view number *mod* replicas). A successful consensus round operates in four distinct phases:

- **Prepare Phase:** The leader of the current view polls its queue for pending client requests. It proposes a new **Node** (block) containing the client's command and cryptographically links it to the hash of the highest known Quorum Certificate (**highQC**). The leader broadcasts a **Prepare** message to all replicas. Upon receiving this, replicas evaluate the *SafeNode* predicate. If the node is deemed safe, the replica replies to the leader with a **PrepareVote** containing its threshold signature share.
- **Pre-Commit Phase:** Once the leader receives $n - f$ **PrepareVote** messages, it combines the signature shares into a **PrepareQC**. It broadcasts this QC inside a **PreCommit** message. Replicas verify the QC's threshold signature and reply with a **PreCommitVote**.
- **Commit Phase:** The leader collects $n - f$ **PreCommitVote** messages to form a **PreCommitQC** and broadcasts it in a **Commit** message. Upon receiving and

verifying this message, replicas update their `lockedQC` to this new QC. This step is crucial for the protocol's safety, as it prevents replicas from voting for conflicting branches in the future. They then reply with a `CommitVote`.

- **Decide Phase:** The leader aggregates $n - f$ `CommitVote` messages into the final `CommitQC` and broadcasts a `Decide` message. Upon receiving this, replicas append the `Node` to their local blockchain, execute the client command, and immediately transition to the next view.

2.3 Client library

Upon initialization, the client is given access to a Client Library, which is how the client communicates with the application. The Client Library has a list containing the addresses of each member of the Blockchain. It proceeds to establish a connection with all of the Blockchain members via APL. Then, it waits for the client to submit a request and sends it to the members, that will attempt to reach consensus and commit it to the Blockchain. The request includes a string sent by the client through the terminal, as well as the client's public key, a signature of the request and a nonce for freshness. The public key is used by the members to verify the signature sent in the message. The members will keep repeating the consensus process until the request is finally executed. Once that happens, they send a reply message to inform the client.

2.4 Smart Contracts

The DepChain contract layer is implemented using the Hyperledger Besu EVM [5] to support both native cryptocurrency (DepCoin) transfers and ERC-20 token (ISTCoin) logic. The system maintains a global "World State" that is persisted in a JSON-based blockchain ledger, containing account balances, nonces, and smart contract storage.

Every transaction in the system has an associated cost paid in DepCoin to prevent resource exhaustion attacks. The `BlockchainExecutor` handles two types of operations:

- Native DepCoin Transfers: These follow a simplified logic where a flat intrinsic fee of 21,000 gas is charged. The executor verifies the sender's balance, decrements the amount and fee, and increments the receiver's balance directly in the world state.
- Smart Contract Calls (ISTCoin): When a client interacts with the ISTCoin contract, the transaction payload is executed by the Besu EVM. The total fee is calculated as $gas_used \times gas_price$, where gas_used is extracted from the EVM's execution trace. Clients specify a gas_limit to cap their potential expenditure, if execution exceeds this limit, the transaction is aborted, but the fee is still deducted to compensate the network.

The `BlockchainStateManager` is responsible for transitioning the system state between blocks.

- **Genesis Initialization:** The system starts from a `genesis.json` file that defines the initial distribution of DepCoins and deploys the ISTCoin smart contract bytecode.
- **Block Appending:** After the HotStuff consensus decides on a transaction order, the `BlockchainExecutor` processes the batch. The resulting state changes (updated balances, nonces, and contract storage slots) are serialized into a new block.
- **Integrity:** Each block stores a `block_hash` calculated from its state and a `previous_block_hash`, creating an immutable chain that replicas use to verify the ledger's integrity.

To fulfill the security requirements against "Approval Frontrunning," we modified the standard ERC-20 `approve` function into `safeApprove`. This function requires the client to submit the `_currentValue` they believe the spender has. If a malicious spender attempts to use the allowance before the update transaction is processed, the `allowance[msg.sender][_spender] == _currentValue` check will fail, causing the transaction to revert and protecting the owner's funds.

To maximize network efficiency and follow the project requirements, transactions within a block are not ordered randomly. The `Transaction` class implements a `compareTo` logic that prioritizes transactions with the highest total fee ($gas_price \times gas_limit$). For transactions with identical fees, they are grouped by sender address and ordered by nonce to ensure sequential execution and state consistency.

3 Threat Assessment

The DepChain system is designed to maintain safety and liveness in an environment where the network is unreliable and a subset of participants may act maliciously. We categorize these threats into server-side and client-side attacks.

3.1 Byzantine Members Threats

We assume that a subset of f blockchain members can behave arbitrarily to disrupt the system and that at least $3f + 1$ replicas exist.

- **Forking:** A malicious leader could attempt to send different blocks to different replicas. We mitigate this through the **Safe Node** check, ensuring a node is only accepted if its parent hash matches the **Locked Quorum Certificate** and the view number is increasing.
- **Request Forgery:** The byzantine leader could also try to change the request sent by the client. To prevent this all replicas must verify the leader's proposed block against the request they also received from the client, since the client's request is broadcast to all replicas.
- **Liveness Attacks:** If a leader stops proposing blocks or ignores messages, because the system uses a timeout-based detector, it will trigger a view change, rotating the leader ensuring the blockchain continues.

- **Block Value Overflow:** A byzantine leader could try to create a block with a total transaction value bigger than what is allowed, so each replica when receiving the leader's proposed block will verify if it's total value is under the limit (80% of 500.000).
- **Quorum Forgery:** To prevent a single Byzantine node from faking a network agreement, we use **Threshold Signature** [3]. The consensus only progresses when a **Quorum Certificate** is formed by $2f + 1$ partial signatures, proving that the majority of the members validated the block.
- **Message Tampering:** To ensure that the sent messages remain intact when sent over UDP, every message is validated against its cryptographic signature and MAC before being processed by the replica.

3.2 Byzantine Client Threats

Starting from the second stage of the project, the client can now be byzantine as well. Here are the threats we considered and handled:

- **Replay Attacks:** An attacker might capture a valid transaction and attempt to re-broadcast, we prevent this by including a nonce in the client's request, all the replicas track the last received nonce for each account and will discard any message that contains an invalid nonce.
- **Signature Forgery:** A client could attempt to issue a transaction from an account they do not own, to prevent this, every request must be signed by the account's private key, and all messages where the signature verification fails are dropped.
- **Approval Frontrunning:** In standard ERC-20 contracts, a malicious spender can "race" a transaction to spend an allowance before a new, lower one is set. Our implementation add verifications to ensure that a `transferFrom` operation cannot be exploited during an allowance update, allowing only the first operation to succeed.

To ensure that no byzantine process proceeds with its faulty behavior as a legitimate process, we have implemented the necessary verifications.

The first integrity mechanism which ensures that the client message hasn't been tampered with, is a MAC that is generated by aggregating the client's public key, the message nonce and the command the client is generating. This is received by each replica which will then verify the MAC legitimacy, dropping any invalid client signatures. Afterwards, when the leader of the current view proposes a block with the client's command, the replicas also verify if the content hasn't been tampered in the case that the leader is byzantine and tries to forge a false command.

Then, the most important mechanism that allows us to verify that the messages sent from each replica are legitimate is the threshold signatures. Threshold signatures allows us to proceed with the consensus even if there are byzantine/faulty processes as long as we follow the minimum number of partial signatures. To calculate the minimum number of signatures we need in order to

proceed with the consensus we have the expression " $2f+1$ " where f represents the number of faults our system is capable of tolerating (e.g. To tolerate 1 faulty/byzantine process we need 3 signatures to guarantee that at least 2 of them come from legitimate processes). This mechanism was implemented by using the library given by the professors (sweis/threshsig) [3] with the small detail of making some library classes implement serializable in order to allow them to be transmitted in the messages.

Finally, we also verify if the node that comes from the message is safe. To be safe, the node's parent hash that comes with the message has to have the hash corresponding to locked quorum certificate node hash and that the current view number is higher than the locked certificate view which corresponds to the previous block. This creates a proof of work chain where the current node depends on the previous node.

4 Tests

To ensure the resilience and correctness of the DepChain network, we developed a comprehensive list of tests that evaluates the system under both ideal scenarios and adversarial conditions. The validation process is divided into two primary categories: Functional and Security Testing, which verifies the integrity of the Ethereum Virtual Machine (EVM) state and cryptographic protocols, and Byzantine Fault Testing, which assesses the consensus mechanism's ability to maintain safety and liveness when faced with byzantine or crashing nodes.

Our testing environment utilizes a network of four blockchain members ($N=4, f=1$) and various client identities. These tests specifically target critical blockchain operations, including multi-currency transfers (DepCoin and ISTCoin), persistent state recovery after member crashes, and the mitigation of smart contract vulnerabilities such as approval front-running.

The following Table 1 detail the specific scenarios simulated, the attack vectors explored, and the system's defensive responses to ensure a highly dependable decentralized ledger.

Test / Behavior	Description and System Defense
Byzantine & Fault Tolerance Tests	
Node Crash	A node stops execution mid-consensus. The network uses a <code>TimeoutHandler</code> to trigger a view change and elect a new leader. (Sec. 3.1)
No Response	A node completes the handshake but drops consensus messages. Replicas proceed once a quorum ($2f + 1$) is reached. (Sec. 3.1)
Random Message	A leader sends malformed bytes. Replicas fail to parse the payload, dropping the message and triggering a timeout. (Sec. 3.1)
Wrong Command	A leader proposes a block with modified or empty commands. Replicas reject the block because the leader cannot forge the client's signature. (Sec. 3.1)
Bad Sorting	A leader violates deterministic gas price sorting. Honest replicas reject the proposal, ensuring priority is given to higher fees. (Sec. 2.4)
Oversized Block	A leader includes transactions exceeding the block gas limit. Replicas verify the total gas and reject the proposal. (Sec. 3.1)
Invalid Signature	The client signs the transaction with a random, fake private key. Replicas verify the signature and drop the transaction. (Sec. 3.2)
Replay Attack	The client sends the same transaction twice with the same nonce. Replicas verify the nonce is duplicated and drop the second transaction. (Sec. 3.2)
Functional, Security & Persistence Tests	
Blockchain Loader	Verifies that the Blockchain can be successfully loaded and attempts to fetch a client's balance. (2.4)
Transaction Sort	Ensures that transactions are sorted according to the established criteria (Fee $\rightarrow$ Sender $\rightarrow$ Nonce). (2.4)
State Persistence	Members save the "World State" to disk. After a crash and restart, the node recovers the latest balances from JSON storage instead of starting from genesis. (2.4)
Session Resumption	If a client restarts, the APL layer performs a new Diffie-Hellman handshake to update session keys and prevent stale connection errors. (3.1)
Threshold Setup	Verifies the KeyStore contains the Master Public Key, RSA private keys, and the mathematical t -of- n shares required for Quorum Certificates. (3.1)
Multi-Currency Transfers	Validates the logic for both native DepCoin and ISTCoin , ensuring balances update correctly and gas fees are deducted. (2.4)
Out of Gas Transfer	The client sets the gas limit too low and the gas used exceeds it. The transaction is aborted and gas fees are still deducted. (2.4)
Unapproved Transfer	Ensures a transferFrom call reverts if the owner has not explicitly granted an allowance to the spender. (3.2)
Approval Frontrunning	Prevents "approval front-running" by requiring the spender's current allowance to be specified before updating to a new value. (3.2)

Table 1. Comprehensive summary of DepChain Byzantine and Functional test suite.

5 Conclusion

Building the 1st stage of DepChain depended on designing and implementing a blockchain system, which we achieved by having N blockchain members (four in this case) perform the Basic HotStuff consensus in order to execute requests made by the client, that communicates with the application via the Client Library. All the communication between processes is done using the Authenticated Perfect Links that our group implemented as an abstraction of UDP sockets for this project. Our system is secure against byzantine behavior for members (up to $(N - 1)/3$ byzantine processes), which includes process crashes, no response to consensus messages and sending fake client values and commands. For the 2nd Stage, the clients can now request transactions, which if valid will be gathered in a block and go through the consensus phase in order to be appended to the blockchain. The clients now also have byzantine behaviour, which includes replay attacks, fake signatures and approval frontrunning, all properly tested to ensure that our security and consensus mechanisms are functioning correctly.

References

1. Yin, M., Malkhi, D., Michael K. , Guy Golan Gueta, and Ittai Abraham, F.: Hot-Stuff: BFT Consensus in the Lens of Blockchain. Journal **(2019)**
2. Matos, M., Romano, P.: Towards a dependable consensus: Basic abstractions.
3. Victor Shoup: Practical Threshold Signatures **(2000)**
4. ERC-20 Token Standard by Ethereum <https://ethereum.org/en/developers/docs/standards/tokens/erc-20>
5. Hyperledger Besu EVM <https://github.com/hyperledger/besu/tree/main>